VOL. 04

React

Components, hooks and the mental model behind modern interfaces

Source	react.dev
Generated	2026-10-10
Level	Intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 04 - React

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 7

Chapter map 7

Chapter 1 - What React is and why it exists 8

1.1 Summary 8

1.2 Concepts 8

What React is, and what it is not 8

The core idea: your UI is a function of state 9

The component tree 9

The toolchain 10

The rules of React 10

1.3 Examples 10

1.4 Common mistakes and best practices 12

1.5 Practice exercises 12

Solutions 13

1.6 Mini test 13

Answer key and explanations 13

Chapter 2 - JSX and rendering elements 13

2.1 Summary 14

2.2 Concepts 14

From JSX to function calls 14

The syntax rules 14

What can go inside curly braces 14

Elements are immutable 15

Rendering nothing 15

2.3 Examples 15

2.4 Common mistakes and best practices 17

2.5 Practice exercises 17

Solutions 17

2.6 Mini test 18

Answer key and explanations 18

Chapter 3 - Components, props and composition 18

3.1 Summary 18

3.2 Concepts . . . 19
Defining and rendering a component . . . 19
Props . . . 19
The `children` prop . . . 19
Composition instead of inheritance . . . 20
One-way data flow . . . 20
Keys . . . 20
3.3 Examples . . . 21
3.4 Common mistakes and best practices . . . 22
3.5 Practice exercises . . . 23
Solutions . . . 23
3.6 Mini test . . . 23
Answer key and explanations . . . 24
Chapter 4 - State with useState . . . 24
4.1 Summary . . . 24
4.2 Concepts . . . 24
Anatomy . . . 24
State is a snapshot . . . 25
Batching . . . 25
State values are replaced, not mutated . . . 25
State versus props . . . 26
The rules of hooks . . . 26
4.3 Examples . . . 26
4.4 Common mistakes and best practices . . . 28
4.5 Practice exercises . . . 29
Solutions . . . 29
4.6 Mini test . . . 29
Answer key and explanations . . . 30
Chapter 5 - Handling events . . . 30
5.1 Summary . . . 30
5.2 Concepts . . . 30
The event props . . . 30
Passing handlers . . . 31
The event object . . . 31
Children notify parents . . . 31

Naming . . . 31
5.3 Examples . . . 32
5.4 Common mistakes and best practices . . . 34
5.5 Practice exercises . . . 34
Solutions . . . 34
5.6 Mini test . . . 34
Answer key and explanations . . . 35
Chapter 6 - Lists and conditional rendering . . . 35
6.1 Summary . . . 35
6.2 Concepts . . . 35
Rendering arrays . . . 35
Choosing keys . . . 36
The conditional patterns . . . 36
The zero trap . . . 36
Filtering and empty states . . . 36
6.3 Examples . . . 37
6.4 Common mistakes and best practices . . . 38
6.5 Practice exercises . . . 39
Solutions . . . 39
6.6 Mini test . . . 39
Answer key and explanations . . . 40
Chapter 7 - Forms and controlled inputs . . . 40
7.1 Summary . . . 40
7.2 Concepts . . . 40
The controlled loop . . . 40
One field or many . . . 40
The input types . . . 41
Submitting . . . 41
Validation . . . 41
7.3 Examples . . . 41
7.4 Common mistakes and best practices . . . 44
7.5 Practice exercises . . . 44
Solutions . . . 45
7.6 Mini test . . . 45
Answer key and explanations . . . 45

Chapter 8 - Side effects with useEffect 45
8.1 Summary 46
8.2 Concepts 46
Where each kind of code belongs 46
Anatomy 46
The three dependency shapes 46
Cleanup 47
Strict Mode 47
Stale values and loops 47
8.3 Examples 47
8.4 Common mistakes and best practices 49
8.5 Practice exercises 50
Solutions 50
8.6 Mini test 50
Answer key and explanations 50
Chapter 9 - Data fetching patterns 51
9.1 Summary 51
9.2 Concepts 51
The request lifecycle 51
The state shape 51
Race conditions 51
Cancellation anatomy 52
When to add a library 52
9.3 Examples 52
9.4 Common mistakes and best practices 55
9.5 Practice exercises 56
Solutions 56
9.6 Mini test 56
Answer key and explanations 57
Chapter 10 - Refs, context, custom hooks and performance 57
10.1 Summary 57
10.2 Concepts 57
Refs 57
Context 58
Memoisation 58

Custom hooks . . . 58
What comes next . . . 59
10.3 Examples . . . 59
10.4 Common mistakes and best practices . . . 62
10.5 Practice exercises . . . 62
Solutions . . . 62
10.6 Mini test . . . 62
Answer key and explanations . . . 63
Cheat sheet . . . 63
11.1 Commands . . . 63
11.2 JSX . . . 63
11.3 Hooks . . . 64
11.4 State recipes . . . 64
11.5 Events . . . 64
11.6 Lists and conditionals . . . 65
11.7 Forms . . . 65
11.8 Effects . . . 65
11.9 Quick answers . . . 66
Final test . . . 66
12.1 Final test answers . . . 67
Sources . . . 68

How to use this volume

Prerequisites. HTML and CSS basics (Vol. 01) and working modern JavaScript (Vol. 03): `let` and `const`, arrow functions, destructuring, template literals, `map` and `filter`, ES modules and `async`/`await`. No React experience is assumed.

Estimated study time. About 30 hours: two to three hours per chapter, plus the time you spend on your own experiments. Chapters 1 to 7 are the core; chapters 8 to 10 cover the parts that make an application real.

How the chapters work. Every chapter opens with a summary, explains the concepts, shows numbered examples with the output they produce, lists the common mistakes with the best practice for each, and ends with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. All React examples live inside a Vite project. Blocks tagged `js` are plain JavaScript you can run with Node; blocks tagged `jsx` are components you paste into a file such as `src/App.jsx`; blocks tagged `text` show what you see on screen; blocks tagged `bash` are terminal commands.

shell

```bash
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

output

```text
  VITE v7.0.0  ready in 431 ms

  ➜  Local:   http://localhost:5173/
  ➜  press h + enter to show help
```

Version numbers change with every release; the shape of the output does not.

About the source. The syllabus follows the official React learning path at react.dev. The project themes used in the exercises are inspired by the ReactPlay catalogue, which is treated only as a source of practice ideas, never as a text to copy.

Chapter map

#	Chapter	You will learn
1	What React is and why it exists	the problems React solves, the Vite toolchain, your first component
2	JSX and rendering elements	JSX syntax, expressions in markup, fragments, escaping
3	Components, props and composition	function components, the props object, children, one-way data flow

#	Chapter	You will learn
4	State with useState	local state, setter functions, snapshots, batching, immutability
5	Handling events	event props, the event object, passing callbacks up
6	Lists and conditional rendering	`map`, keys, `&&`, ternaries, early returns
7	Forms and controlled inputs	controlled inputs, one state object, submit and validation
8	Side effects with useEffect	synchronising with the outside world, dependencies, cleanup
9	Data fetching patterns	fetch inside effects, loading and error states, race conditions
10	Refs, context, custom hooks and performance	`useRef`, context, extracting hooks, memoisation
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - What React is and why it exists

1.1 Summary

- React is a JavaScript library for describing user interfaces as components.
- A component is a function that returns the markup for one part of the screen.
- Instead of editing the DOM by hand, you describe the screen for each state and React performs the update.
- Components compose: a page is a tree of functions that pass data down through props.
- Hooks such as `useState` and `useEffect` add memory and side effects to function components.
- The browser only needs the bundled JavaScript; Node.js and Vite are development tools.
- `npm create vite` scaffolds a working app with a dev server, JSX support and hot reload.
- The entry point `src/main.jsx` mounts the component tree into one DOM node described by `index.html`.
- React itself is small; routing, data fetching and global state come from extra libraries.
- Everything in this volume follows the official learning path published at react.dev.

1.2 Concepts

What React is, and what it is not

React has one job: keep what you see on screen in sync with your data. You describe the interface, React changes the document. Concerns that surround the view are deliberately left to the ecosystem:

Concern	React's answer	Usual library
Views	components	-
Local state	`useState`, `useReducer`	-
Routing	-	React Router
Server data	-	TanStack Query
Global state	context, reducers	Redux, Zustand
Testing	-	Vitest, Testing Library

The core idea: your UI is a function of state

A component maps data to markup. When the data changes, React calls the component again and compares the result with what is on screen.

output

```
   state              render                screen
  +--------+         +-----------+         +---------+
  | count  | ------> | f(state)  | ------> | shows 3 |
  +--------+         +-----------+         +---------+
      ^                                        |
      |            a click calls setCount(4)   |
      +----------------------------------------+
```

You never write "find the element and change it". You write "this is what the screen looks like when the count is 4".

The component tree

An application is a tree of components. Every component is a small function with a narrow job:

output

```
<App>
  <Header>
    <SearchBar />
  </Header>
  <Sidebar>
    <NavItem />
    <NavItem />
  </Sidebar>
  <Main>
    <TaskList>
      <TaskItem />
      <TaskItem />
    </TaskList>
  </Main>
</App>
```

Data flows down the tree as props. Events travel back up as function calls.

The toolchain

Tool	Job
Node.js	runs the dev server and the build
npm	installs packages listed in `package.json`
Vite	dev server with hot reload, and production bundler
JSX compiler	built into Vite; turns JSX into function calls
Browser	runs the bundled JavaScript React produces

The rules of React

Three rules keep the model predictable:

1. Render must be pure: the same props and state always produce the same markup.
2. Never mutate props or state; create new values instead.
3. Hooks are called at the top level of a component, never inside conditions.

1.3 Examples

Example 1: the manual way, before React.

JavaScript

```javascript
// Building one list item without React.
const li = document.createElement("li");
li.textContent = "Buy milk";
document.querySelector("ul").append(li);
```

output

```
(no console output; a <li>Buy milk</li> element is appended to the first <ul>)
```

Adding a second item, removing the first, and keeping classes in sync means writing all of that again, by hand, every time.

Example 2: the React way.

JSX

```jsx
function ShoppingList() {
  const items = ["Buy milk", "Walk the dog"];
  return (
    <ul>
      {items.map((item) => (
        <li key={item}>{item}</li>
      ))}
    </ul>
  );
}
```

output

```
- Buy milk
- Walk the dog
```

The DOM React produced:

output

```
<ul><li>Buy milk</li><li>Walk the dog</li></ul>
```

Example 3: scaffold a project with Vite.

shell

```
npm create vite@latest my-app -- --template react
```

output

```
Need to install the following packages:
  create-vite@7.0.0
Ok to proceed? (y) y

Scaffolding project in C:\projects\my-app...

Done. Now run:

  cd my-app
  npm install
  npm run dev
```

(The output is trimmed; the version numbers differ on every machine.)

Example 4: install and start the dev server.

shell

```
cd my-app
npm install
npm run dev
```

output

```
added 214 packages in 12s

  VITE v7.0.0  ready in 431 ms

  ➜  Local:   http://localhost:5173/
  ➜  press h + enter to show help
```

Open the printed address and the browser shows whatever `App` returns.

Example 5: the entry point you get from the template.

JSX

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.querySelector("#root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

output

```
(the page from index.html shows the default App: a Vite + React heading,
two links and a counter button)
```

`StrictMode` is a development-only wrapper that calls your components twice so that unsafe code fails loudly while you write it.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Calling a component as a function (`App()` in JSX)	loses state and hooks	render it as `<App />`
Editing the DOM behind React's back	React overwrites it	describe the UI, use refs for imperative needs
Expecting routing or fetching to be built in	you rebuild buggy versions	add React Router or TanStack Query
Using Node-only APIs in components	blank page in the browser	keep Node in the toolchain
One giant component	unreadable, nothing is reusable	split into small components
Installing packages globally	version drift between projects	always `npm install` inside the project

- Keep each component small enough to read without scrolling.
- When something looks wrong, open the browser console before anything else.

1.5 Practice exercises

1. Which three commands create, install and start a Vite React project?
2. Where does React mount the application, and which file performs the mounting?
3. A component returns two sibling elements. Name two ways to fix it.
4. Why is it wrong to write `user.name = "Ada"` on a props object?
5. React handles views only. Name three concerns you must add yourself.

Solutions

1. `npm create vite@latest my-app -- --template react`, then `cd my-app`, then `npm install`, then `npm run dev`.
2. Into the element that `createRoot` receives, by default `#root` from `index.html`; the mounting code lives in `src/main.jsx`.
3. Wrap them in a fragment (`<>...</>`) or in one parent element such as a `div`.
4. Props are read-only: the object is owned by the parent, and mutating it breaks the one-way data flow and confuses React's change detection.
5. Any three of: routing, HTTP requests and caching, global state, form validation, testing, animation, internationalisation.

1.6 Mini test

1. What does a React component return? A) A real DOM node B) A description of what the screen should show C) An HTML string D) A promise
2. Which command scaffolds a React project with Vite? A) `npm start react` B) `node create app` C) `npm create vite@latest my-app -- --template react` D) `git clone react`
3. How does a child tell its parent that something happened? A) By mutating the parent's state variable B) By calling a function the parent passed as a prop C) By dispatching a native DOM event on the parent D) By rewriting the parent's HTML
4. True or false: a production site needs Node.js on the server for the browser to render React components.
5. Which file mounts the component tree in a default Vite template? A) `index.html` on its own B) `src/App.jsx` C) `src/main.jsx` D) `vite.config.js`
6. Why must the render phase stay pure? A) It makes the bundle smaller B) React may run it more than once with the same inputs and expects the same result C) Browsers require it D) It is a naming convention

Answer key and explanations

1. B - a component returns an element tree, an immutable description; React turns elements into DOM nodes.
2. C - `npm create vite@latest` with the react template creates and configures a working project in one command.
3. B - children call function props; the state itself always stays in the parent that owns it.
4. False - Node runs the build and the dev tools; the browser only receives and executes the bundled script.
5. C - `src/main.jsx` calls `createRoot(...).render(<App />)`.
6. B - Strict Mode and concurrent rendering can re-run render, so render must not cause side effects.

Chapter 2 - JSX and rendering elements

2.1 Summary

- JSX lets you write markup inside JavaScript files.
- A compiler, built into Vite, turns JSX into function calls before the browser sees it.
- A component returns one root element, or several wrapped in a fragment.
- Every tag must be closed, including `<img />` and `<br />`.
- HTML attributes are written as React props: `className`, `htmlFor`, and camelCase event props.
- Any JavaScript expression can be embedded with curly braces.
- The `style` prop takes an object with camelCased property names, not a string.
- Values are escaped, so JSX protects you from injecting raw HTML.
- JSX elements are immutable objects, and a new render produces new elements.
- Comments inside JSX are written as `{/* ... */}`.

2.2 Concepts

From JSX to function calls

JSX

```jsx
const title = <h1 className="greeting">Hello, Ada</h1>;
```

With the automatic runtime this becomes, in simplified form:

JSREF

```jsref
import { jsx as _jsx } from "react/jsx-runtime";
const title = _jsx("h1", { className: "greeting", children: "Hello, Ada" });
```

An element is a plain object with a `type`, some `props` and a `key`. React reads those objects to build the DOM and to update it later.

The syntax rules

Rule	Wrong	Right
One root element	`return <h1 /><p />`	wrap in a `div` or in `<>`
Close every tag	`<img src="a.png">`	`<img src="a.png" />`
Class attribute	`class="a"`	`className="a"`
Label attribute	`for="a"`	`htmlFor="a"`
Inline style	`style="color: red"`	`style={{ color: "red" }}`
Comments	`// note` inside markup	`{/* note */}`

What can go inside curly braces

Curly braces accept exactly one expression:

Kind	Example
Variable	`{name}`
Computed value	`{price * count}`
Function call	`{formatPrice(total)}`
Conditional	`{isOpen ? "Open" : "Closed"}`
List of elements	`{items.map((i) => <li key={i}>{i}</li>)}`

Statements do not work: `if`, `for` and `const` belong above the `return`, not inside the markup.

Elements are immutable

You cannot change an element after creating it. To change the screen you build a new element with new data, and React compares the old and new descriptions.

Rendering nothing

Returning `null` from a component, or putting `null` in the markup, renders nothing at all.

2.3 Examples

Example 1: expressions inside markup.

JSX

```jsx
function PriceTag({ name, price, inStock }) {
  return (
    <p>
      {name}: {price > 0 ? `$${price.toFixed(2)}` : "free"}{" "}
      {inStock ? "(in stock)" : "(sold out)"}
    </p>
  );
}
```

Rendered with `name="Coffee"`, `price={3.5}`, `inStock={true}`:

output

```
Coffee: $3.50 (in stock)
```

The `{" "}` produces the space between the two expressions.

Example 2: attributes and inline styles.

JSX

```jsx
function Badge({ label }) {
  const style = {
    color: "white",
    backgroundColor: "rebeccapurple",
    padding: "2px 8px",
  };
  return (
    <span className="badge" style={style}>
      {label}
    </span>
  );
}
```

output

```
<span class="badge" style="color: white; background-color: rebeccapurple; padding: 2px 8px;">New</span>
```

The outer braces of `style={{ ... }}` open a JavaScript expression; the inner braces are the object literal itself.

Example 3: fragments.

JSX

```jsx
function Field({ label, hint }) {
  return (
    <>
      <dt>{label}</dt>
      <dd>{hint}</dd>
    </>
  );
}
```

output

```
<dt>Email</dt><dd>We never share it</dd>
```

No wrapper element appears in the DOM; the fragment exists only in the source.

Example 4: escaping.

JSX

```jsx
const comment = '<img src="x" onerror="alert(1)">';

function Comment() {
  return <p>{comment}</p>;
}
```

output

```
<img src="x" onerror="alert(1)">
```

The paragraph shows the text itself. Nothing is parsed as HTML, which is why JSX is safe by default against injection.

Example 5: element identity.

JavaScript

```
const a = { type: "h1", props: { children: "Hi" } };
const b = { type: "h1", props: { children: "Hi" } };
console.log(a === b);
console.log(a.type === b.type && a.props.children === b.props.children);
```

output

```
false
true
```

Two elements are never the same object. React compares their type and props to decide what to change.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Two roots without a fragment	the file does not compile	wrap in `<>` or one parent
`class` instead of `className`	the attribute is ignored, styles vanish	use React prop names
`style="color: red"`	React rejects a string here	object with camelCased keys
`if` or `for` inside braces	braces hold one expression only	compute the value before `return`
Expecting a string of HTML to render	the tags appear as text	write markup, or sanitise carefully
Forgetting `{" "}` between expressions	words run together	add an explicit space

- Read the compiler error message: it always names the line and the rule.
- Keep markup flat; extract a component instead of nesting deeply.

2.5 Practice exercises

1. Rewrite `<div class="box" for="name">` with valid React props.
2. Why does `{items.map(...)}` work while `{for (...) ...}` does not?
3. What is wrong with `<img src="a.png">`?
4. In `style={{ backgroundColor: "red" }}`, why are there two levels of braces?
5. How do you add a comment inside returned JSX?

Solutions

1. `<div className="box" htmlFor="name">`.
2. Curly braces hold one expression; `map` is an expression and returns an array, while `for` is a statement. Do the loop before the `return`.
3. The tag is not closed: `<img src="a.png" />`.

4. The outer braces are the JSX expression, the inner braces are the style object literal.
5. `{/* note */}`.

2.6 Mini test

1. What does the JSX compiler produce? A) Plain HTML text B) JavaScript function calls that create objects C) CSS rules D) SQL statements
2. Which attribute name is invalid in JSX? A) `className` B) `htmlFor` C) `class` D) `tabIndex`
3. How is a component with two sibling elements written? A) Separated by a comma B) As an array with keys C) Wrapped in a fragment or one parent element D) Separated by a semicolon
4. What may appear inside curly braces? A) Any JavaScript expression B) Statements such as `if` and `for` C) Raw HTML tags D) CSS declarations
5. True or false: the `style` prop accepts a CSS string such as `"color: red"`.
6. What does `{"<b>hi</b>"}` render? A) Bold text B) The literal text, escaped C) Nothing D) A runtime error

Answer key and explanations

1. B - the build turns JSX into calls to `jsx`/`createElement` that produce element objects.
2. C - HTML attribute names are replaced by React prop names, and `class` is not one of them.
3. C - JSX needs a single root; fragments group siblings without adding a DOM node.
4. A - a single expression, evaluated to text or to elements.
5. False - `style` takes an object with camelCased property names.
6. B - values are escaped as text, which blocks script injection.

Chapter 3 - Components, props and composition

3.1 Summary

- A component is a function whose name starts with a capital letter and that returns JSX.
- React calls the component with a single props object.
- Lower-case tags in JSX are treated as HTML elements.
- Props are read-only inside the component that receives them.
- Destructuring the props keeps the signature short and self-documenting.
- Default values come from destructuring defaults, never from mutation.
- `children` carries whatever JSX the parent placed between the tags.
- Composition replaces inheritance: general components are given specific children.
- Siblings communicate only through the parent they share.

- Keys give React a stable identity for every child in a list of elements.

3.2 Concepts

Defining and rendering a component

JSX

```jsx
function Avatar({ url, alt }) {
  return <img src={url} alt={alt} className="avatar" />;
}

// an arrow function works exactly the same way
const Avatar = ({ url, alt }) => <img src={url} alt={alt} className="avatar" />;
```

The capital letter matters:

Tag	Meaning
`<Avatar />`	your component, called with props
`<avatar />`	an HTML element, rendered as-is

`export default function App()` makes the component importable from another file; `export function Card()` allows several named components per file.

Props

Props arrive as one object, and they belong to the parent:

JSX

```jsx
function PriceTag({ value, currency = "USD" }) {
  return <span>{currency} {value}</span>;
}

<PriceTag value={19} currency="EUR" />   // expression as a value
<PriceTag value={19} />                  // uses the default
```

Passed	Example
a value	`<PriceTag value={19} />`
text	`<PriceTag value={19} currency="EUR" />`
markup	`<Card><p>Hello</p></Card>`
a function	`<LikeButton onLike={handleLike} />`

Mutating a prop, writing to it, or storing it somewhere for later is a bug waiting to happen.

The `children` prop

Whatever a parent writes between the opening and closing tags arrives as `children`:

JSX

```jsx
function Card({ title, children }) {
  return (
    <section className="card">
      <h2>{title}</h2>
      {children}
    </section>
  );
}

<Card title="Inbox">
  <p>You have two unread messages.</p>
</Card>
```

Composition instead of inheritance

React has no base component to extend. You build specialised components from general ones:

General component	Specialised by
`Panel`	different `children` and a title
`Button`	a different `variant` prop
`Row`	different cells
`List`	different items and a render function

A component can also be passed as a prop when a slot must be filled from outside:

JSX

```jsx
function Layout({ sidebar, children }) {
  return (
    <div className="page">
      <aside>{sidebar}</aside>
      <main>{children}</main>
    </div>
  );
}

<Layout sidebar={<Nav />}>
  <Inbox />
</Layout>
```

One-way data flow

output

```
parent state
     |
     |  props (flow down)
     v
  <Child />  --calls onSomething-->  parent setState  -->  new props
```

The child never touches the parent's state. It asks, by calling a function the parent gave it.

Keys

When you render a list of elements, each one needs a `key`. Chapter 6 covers the details; the rule is unique among siblings and stable over time.

3.3 Examples

Example 1: props with defaults.

JSX

```jsx
function Avatar({ url, alt, size = 48 }) {
  return (
    <img
      src={url}
      alt={alt}
      width={size}
      height={size}
      className="avatar"
    />
  );
}

<Avatar url="/ada.png" alt="Ada Lovelace" />
<Avatar url="/alan.png" alt="Alan Turing" size={64} />
```

output

```
<img src="/ada.png" alt="Ada Lovelace" width="48" height="48" class="avatar">
<img src="/alan.png" alt="Alan Turing" width="64" height="64" class="avatar">
```

Example 2: children.

JSX

```jsx
function Card({ title, children }) {
  return (
    <section className="card">
      <h2>{title}</h2>
      {children}
    </section>
  );
}

<Card title="Inbox">
  <p>You have two unread messages.</p>
</Card>
```

output

```
<section class="card">
  <h2>Inbox</h2>
  <p>You have two unread messages.</p>
</section>
```

Example 3: composition.

JSX

```jsx
function Page({ sidebar, children }) {
  return (
    <div className="page">
      <header className="page-header"><h1>Inbox</h1></header>
      <aside className="sidebar">{sidebar}</aside>
      <main className="content">{children}</main>
    </div>
  );
}

<Page sidebar={<nav>Home<br />Archive</nav>}>
  <p>No message selected</p>
</Page>
```

output

```
<div class="page">
  <header class="page-header"><h1>Inbox</h1></header>
  <aside class="sidebar"><nav>Home<br />Archive</nav></aside>
  <main class="content"><p>No message selected</p></main>
</div>
```

Example 4: a child that notifies its parent.

JSX

```jsx
function LikeButton({ likes, onLike }) {
  return <button onClick={onLike}>Like ({likes})</button>;
}

function Post() {
  const [likes, setLikes] = useState(1);
  return <LikeButton likes={likes} onLike={() => setLikes(likes + 1)} />;
}
```

After two clicks the button reads:

output

```
Like (3)
```

Example 5: two instances are independent.

JSX

```jsx
function Greeting({ name }) {
  return <h2>Hello, {name}</h2>;
}

<Greeting name="Ada" />
<Greeting name="Alan" />
```

output

```
<h2>Hello, Ada</h2>
<h2>Hello, Alan</h2>
```

Each call receives its own props object; nothing leaks between them.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Mutating a props object or array	the parent sees the change, React does not	create a new value and hand it down
A component called `panel`	rendered as an unknown HTML tag	capitalise component names
Inheriting from another component	React has no inheritance model	compose with children and props
Reading a prop that does not exist	silent `undefined` in the output	destructure with defaults, add TypeScript
A component doing five unrelated jobs	hard to reuse and to test	one job per component
Logic hidden inside JSX	unreadable markup	name functions for each part

- Name components after what they show: `TaskList`, `SearchBar`, `PriceTag`.
- If two components need the same data, move the data to their common parent.

3.5 Practice exercises

1. Why does `<avatar />` not render your component?
2. Rewrite `function PriceTag(props)` so the signature uses destructuring with a default of `"USD"` for `currency`.
3. What is `children`, and where does it come from?
4. A `TaskList` and a `TaskFilter` both need the tasks array. Where should the array live?
5. Give two reasons to prefer composition over inheritance in React.

Solutions

1. JSX treats a lower-case tag as an HTML element; component names must start with a capital letter.
2. `function PriceTag({ value, currency = "USD" }) { ... }`.
3. `children` is an ordinary prop holding the JSX the parent wrote between the component's opening and closing tags.
4. In the closest common parent, which then passes it down as a prop to both.
5. React has no inheritance machinery, and composition makes data flow explicit: the parent owns the data, the children stay generic and reusable.

3.6 Mini test

1. What does React pass to a function component? A) The DOM element it belongs to B) A single props object C) The parent component D) The global store
2. Which tag name renders an HTML element rather than your component? A) `<Avatar />` B) `<avatar />` C) `Avatar()` D) `<AvatarAvatar />`

3. Which statement about props is true? A) A component may rewrite its props B) Props are read-only inside the receiving component C) Props are stored in the DOM D) Props are shared by all instances
4. What does the `children` prop contain? A) The component's state B) The JSX placed between the component's tags C) The component's key D) The parent's name
5. Two sibling components need the same data. What is the recommended fix? A) A global variable B) Lift the data into their common parent and pass props C) Import one component into the other D) Store it in the DOM
6. True or false: React components are usually specialised through class inheritance.

Answer key and explanations

1. B - all attributes and children are collected into one object.
2. B - lower-case tags are HTML; capitalised tags are your components.
3. B - mutating props breaks the one-way flow and hides changes from React.
4. B - children are just another prop, filled by the parent's markup.
5. B - lifting state up keeps a single source of truth in the parent.
6. False - components compose with props and children instead.

Chapter 4 - State with useState

4.1 Summary

- `useState(initial)` returns the current value and a setter function.
- The state value stays fixed for the whole render: it is a snapshot.
- Calling the setter schedules a new render with the next value.
- Use the updater form when the next value depends on the previous one.
- Objects and arrays in state are replaced, never mutated.
- A component may hold as many independent state values as it needs.
- State is private to each component instance.
- Anything computable from props or state is computed, not stored.
- Updates inside one event handler are batched into a single render.
- Hooks must be called at the top level of a component.

4.2 Concepts

Anatomy

JSX

```jsx
const [count, setCount] = useState(0);
```

Part	Meaning
argument	the initial value, or a function that returns it
first item	the value during this render
second item	the setter: `setCount(5)` or `setCount((c) => c + 1)`

A lazy initialiser runs only on the first render:

JSX

```jsx
const [items] = useState(() => readItemsFromDisk());
```

State is a snapshot

During one render, `count` never changes, even after you call the setter. The setter only asks React to run the component again:

JSX

```jsx
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);       // schedules 1
    console.log(count);        // still 0: this render's snapshot
  }

  return <button onClick={handleClick}>{count}</button>;
}
```

When several updates depend on the previous value, use the updater form. Each updater runs in order during the next render:

JSX

```jsx
setCount((c) => c + 1);   // 0 -> 1
setCount((c) => c + 1);   // 1 -> 2
```

Batching

Setters called in the same event handler produce one render, not two:

JSX

```jsx
function handleSubmit() {
  setName("Ada");
  setRole("admin");
}   // a single re-render shows both new values
```

State values are replaced, not mutated

Do this	Not this
`setUser({ ...user, name: "Ada" })`	`user.name = "Ada"`

Do this	Not this
`setItems([...items, newItem])`	`items.push(newItem)`
`setItems(items.filter((i) => i.id !== id))`	`items.splice(index, 1)`

React compares references with `Object.is`, so a mutated object looks unchanged and the screen never updates.

State versus props

	props	state
Comes from	the parent	the component itself
Changes when	the parent re-renders	the setter is called
Writable	no	only through the setter
Scope	one component	one component instance

The rules of hooks

1. Call hooks at the top level, never inside `if`, `for` or nested functions.
2. Call them only from components or from custom hooks (chapter 10).
3. The order must be identical on every render.

4.3 Examples

Example 1: a counter.

JSX

```jsx
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <button onClick={() => setCount(count - 1)}>-</button>
      <span> {count} </span>
      <button onClick={() => setCount(count + 1)}>+</button>
    </div>
  );
}
```

After clicking `+` once:

output

```
0   ->   1
```

(The zero is the initial render, the 1 after the click.)

Example 2: two updaters, one click.

JSX

```jsx
function Double() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => {
      setCount((c) => c + 1);
      setCount((c) => c + 1);
    }}>
      Count: {count}
    </button>
  );
}
```

output

```
Count: 0   (one click)   Count: 2
```

Both queued updaters run during the next render.

Example 3: batching.

JSX

```jsx
function Swap() {
  const [first, setFirst] = useState("a");
  const [second, setSecond] = useState("b");
  console.log(`render: ${first} ${second}`);
  return <button onClick={() => { setFirst("b"); setSecond("a"); }}>Swap</button>;
}
```

output

```
render: a b        (initial)
render: b a        (after one click, printed once)
```

Example 4: an object in state.

JSX

```jsx
function ProfileForm() {
  const [profile, setProfile] = useState({ name: "Ada", email: "" });

  function handleChange(event) {
    const { name, value } = event.target;
    setProfile({ ...profile, [name]: value.trim() });
  }

  return (
    <form>
      <input name="name" value={profile.name} onChange={handleChange} />
      <input name="email" value={profile.email} onChange={handleChange} />
      <p>{profile.name} - {profile.email}</p>
    </form>
  );
}
```

Typing `ada@example.com` into the email field shows:

output

```
Ada - ada@example.com
```

Example 5: an array in state.

JSX

```jsx
function Todos() {
  const [todos, setTodos] = useState([{ id: 1, text: "Learn JSX" }]);

  function add() {
    setTodos([...todos, { id: Date.now(), text: "Learn state" }]);
  }
  function remove(id) {
    setTodos(todos.filter((t) => t.id !== id));
  }

  return (
    <div>
      <button onClick={add}>Add</button>
      <ul>
        {todos.map((t) => (
          <li key={t.id}>
            {t.text} <button onClick={() => remove(t.id)}>x</button>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

After `Add`, then clicking the first `x`:

output

```
- Learn state x
```

Example 6: derive, do not store.

JSX

```jsx
function Stats({ items }) {
  const done = items.filter((i) => i.done).length;
  return (
    <p>{done} of {items.length} done, {items.length - done} remaining</p>
  );
}
```

With `items = [{ done: true }, { done: false }]`:

output

```
1 of 2 done, 1 remaining
```

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`user.name = "Ada"` then assume the screen updates	no re-render, stale screen	`setUser({ ...user, name: "Ada" })`

Mistake	Why it hurts	Best practice
`setCount(count + 1)` twice in one handler	the second uses the stale value	the updater form
Copying a computable value into state	two sources of truth	compute during render
Hooks inside `if` or `for`	hook order breaks, React throws	top level only
`useState` inside a `map`	the number of hooks changes per render	keep one state holding an array
Passing the initial value a fresh object on every render	the state resets? No: it is ignored	fine, but use a lazy initialiser for expensive work

- Only two things belong in state: values the component remembers between renders, and values React must re-render for.
- When a value can be computed from props or other state, compute it.

4.5 Practice exercises

1. Why does `count = count + 1` do nothing to the screen?
2. `const [user, setUser] = useState({ name: "Ada", age: 36 })`. Change the age correctly.
3. Add `"milk"` to the array `items` held in state without mutating it.
4. How many renders happen when one handler calls `setName` and `setRole`, and how do both updates survive?
5. Which two kinds of values should live in state at all?

Solutions

1. Assigning to the local variable only changes the current render's snapshot; React is never asked to render again.
2. `setUser({ ...user, age: 37 })`.
3. `setItems([...items, "milk"])`.
4. One render. Both setters are queued and applied together, so the next render sees both new values.
5. Values the component must remember between renders, and values whose change must update the screen.

4.6 Mini test

1. What does `useState` return? A) The value only B) The value and a setter function C) An object with `get` and `set` D) A ref object
2. What is the value of `count` immediately after `setCount(count + 1)` in the same handler? A) The new value B) The value from the current render C) `undefined` D) It throws

3. Which update is reliable when called twice in a row? A) `setCount(count + 1)` twice B) `setCount((c) => c + 1)` twice C) `count += 2` D) `setCount(count++)`
4. How do you add an item to an array in state? A) `items.push(newItem)` B) `setItems([...items, newItem])` C) `items[items.length] = newItem` D) `setItems(items.concat)` without an argument
5. Two `setState` calls in one event handler cause how many renders? A) Two B) One, batched C) Zero D) One per millisecond
6. True or false: a component instance's state is shared with other instances of the same component.

Answer key and explanations

1. B - the first item is the current value, the second the function that schedules an update.
2. B - state is a snapshot for the duration of the render.
3. B - updaters queue in order and each receives the latest value.
4. B - React detects change by reference, so a new array is required.
5. B - updates inside one event are batched into a single render.
6. False - each instance owns its own state.

Chapter 5 - Handling events

5.1 Summary

- React event props are camelCase: `onClick`, `onChange`, `onSubmit`.
- You pass the handler function itself, not the result of calling it.
- Handlers receive an event object with a consistent interface in every browser.
- `event.preventDefault()` cancels the browser's built-in reaction to an event.
- Arguments are passed with an inline arrow: `onClick={() => remove(id)}`.
- Children notify parents by calling a function prop.
- Handler names start with `handle` and describe what they do.
- React events bubble through the component tree, as DOM events do.
- `stopPropagation` stops bubbling, but lifting state is usually the better fix.
- Keyboard events such as `onKeyDown` support the Escape and Enter patterns.

5.2 Concepts

The event props

DOM attribute	React prop	Note
`onclick`	`onClick`	any clickable element
`onchange`	`onChange`	fires on each keystroke in inputs

DOM attribute	React prop	Note
`onsubmit`	`onSubmit`	put it on the `<form>`
`onkeydown`	`onKeyDown`	Escape and Enter patterns
`onfocus` / `onblur`	`onFocus` / `onBlur`	in React these bubble, unlike the native focus events

CamelCase is mandatory: `onclick` in JSX is just an unknown attribute.

Passing handlers

JSX

```jsx
<button onClick={handleClick}>Save</button>
<button onClick={() => removeTodo(id)}>Delete</button>
```

The first form passes a reference. The second creates a small function so the call can receive an argument. Writing `onClick={handleClick()}` would call the function during render, which is almost never what you want.

The event object

Member	Use
`event.target`	the element the event happened on
`event.currentTarget`	the element the handler is attached to
`event.preventDefault()`	cancels the default browser behaviour
`event.stopPropagation()`	stops the event from bubbling further

React wraps the native event, so the interface behaves the same everywhere.

Children notify parents

JSX

```jsx
function LikeButton({ likes, onLike }) {
  return <button onClick={onLike}>Like ({likes})</button>;
}

function Post() {
  const [likes, setLikes] = useState(1);
  return <LikeButton likes={likes} onLike={() => setLikes(likes + 1)} />;
}
```

output

```
parent state (likes)  --prop-->  <LikeButton />
<LikeButton />         --calls-->  onLike()  ->  setLikes  ->  new prop
```

Naming

Position	Name	Example
prop on a child	`onX`	`onLike`, `onSelect`
function in the parent	`handleX`	`handleLike`, `handleSelect`

5.3 Examples

Example 1: a click handler.

JSX

```jsx
function Welcome() {
  function handleClick() {
    console.log("clicked");
  }
  return <button onClick={handleClick}>Say hello</button>;
}
```

output

```
clicked        (printed once per click)
```

Example 2: a handler with an argument.

JSX

```jsx
function TodoList({ todos, onRemove }) {
  return (
    <ul>
      {todos.map((t) => (
        <li key={t.id}>
          {t.text} <button onClick={() => onRemove(t.id)}>x</button>
        </li>
      ))}
    </ul>
  );
}
```

Clicking the `x` of the first row calls `onRemove(1)` and that row disappears. Clicking another row calls `onRemove` with that row's id, never with `1`.

Example 3: a form that does not reload the page.

JSX

```
function Search() {
  const [query, setQuery] = useState("");

  function handleSubmit(event) {
    event.preventDefault();
    console.log(`submitted: ${query}`);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={query} onChange={(e) => setQuery(e.target.value)} />
      <button type="submit">Go</button>
    </form>
  );
}
```

Typing `react` and submitting prints:

output

```
submitted: react
```

The address bar does not change and the page does not reload.

Example 4: mirroring an input.

JSX

```
function Mirror() {
  const [text, setText] = useState("");
  return (
    <div>
      <input onChange={(e) => setText(e.target.value)} />
      <p>{text}</p>
    </div>
  );
}
```

Typing `ada` one character at a time shows:

output

```
(empty) -> a -> ad -> ada
```

Example 5: Escape closes a panel.

JSX

```
function Panel({ title, onClose }) {
  return (
    <div tabIndex={0} onKeyDown={(e) => { if (e.key === "Escape") onClose(); }}>
      <h3>{title}</h3>
      <p>Press Escape to close.</p>
    </div>
  );
}
```

output

```
(pressing Escape while the panel has focus removes it from the screen)
```

`e.key` is a readable string: `"Enter"`, `"Escape"`, `"ArrowUp"`.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`onClick={handleClick()}`	runs during render, can loop forever	pass the reference
Missing `preventDefault` on submit	the page reloads and state is lost	call it first in the handler
New inline function for every click on a huge list	re-renders memoised children	`useCallback` (chapter 10)
Reaching for `stopPropagation` to fix shared state	logic spreads across components	lift the state up
Mutating the DOM in a handler	fights with React's rendering	change state instead
Inline arrows for complex handlers	markup becomes unreadable	define named functions

- Name the prop `onX` and the handler `handleX`; the pair reads like a sentence.
- Keep handlers small: read the event, call a setter or a callback.

5.5 Practice exercises

1. Why is `onClick={handleClick()}` a bug?
2. Which call stops a form from reloading the page?
3. How do you pass the id of the clicked row to a handler?
4. What is the React prop for the DOM `onchange` attribute on an input?
5. Two sibling components react to the same click. What is the recommended fix?

Solutions

1. The function runs on every render instead of on the click, and the value passed to `onClick` becomes its return value.
2. `event.preventDefault()` inside the `onSubmit` handler.
3. Use an inline arrow: `onClick={() => handleRemove(row.id)}`.
4. `onChange`.
5. Lift the shared state into the parent and let both children read it as props.

5.6 Mini test

1. Which prop is correct for a click? A) `onclick` B) `onClick` C) `ONCLICK` D) `clicked`
2. What is passed to `onClick`? A) The result of calling the handler B) A reference to the handler function C) A string with the function's name D) The event itself

3. Which call prevents a form submission from reloading the page? A) `event.stopPropagation()` B) `event.preventDefault()` C) `return false` at the top of the component D) `event.target.reset()`
4. How do you read which key was pressed in `onKeyDown`? A) `event.key` B) `event.keyCode` C) `event.char` D) `event.name`
5. True or false: a child component can directly change its parent's state.
6. What is the idiomatic name pair for a callback prop and its handler? A) `doThing` and `thing()` B) `onThing` and `handleThing` C) `setThing` and `getThing` D) `click` and `clicked`

Answer key and explanations

1. B - React event props are camelCase.
2. B - React invokes the function and passes the event object to it.
3. B - `preventDefault` cancels the browser's default reaction.
4. A - `event.key` gives a readable name such as `"Escape"`.
5. False - the child calls a function prop; only the parent owns the state.
6. B - `onX` for the prop, `handleX` for the function that reacts to it.

Chapter 6 - Lists and conditional rendering

6.1 Summary

- Arrays are rendered with `map`, which returns one element per item.
- Every item in a rendered list needs a `key` that is unique and stable.
- Prefer data ids for keys; indexes suit only static lists.
- Filter and map during render to turn data into elements.
- `&&` renders its right side only when the left side is truthy.
- Guard `&&` against zero: write `count > 0 && <Badge />`.
- Ternaries choose between two blocks of markup.
- Early returns handle loading and error states cleanly.
- A component may return `null` to render nothing.
- Every list needs an empty state for the moment it has no rows.

6.2 Concepts

Rendering arrays

JSX

```jsx
<ul>
  {people.map((person) => (
    <li key={person.id}>{person.name}</li>
  ))}
</ul>
```

Choosing keys

Key	Verdict
the item's id from your data	best
the array index	only for static lists that never reorder or filter
the displayed text	breaks with duplicates
a random number	breaks identity on every render

Keys are internal to React: a component cannot read its own key through props. React matches old and new children by type and key:

output

```
old: [A#1][B#2][C#3]        new: [B#2][A#1][C#3]
       React keeps the elements, moves them, and only patches what changed.
```

With index keys, React assumes position 0 always holds the same item, so filtering a list can attach state and DOM nodes to the wrong rows.

The conditional patterns

Pattern	Use it for	Example
`&&`	show or hide one thing	`{error && <p>{error}</p>}`
ternary	either of two blocks	`{open ? <Menu /> : <Button />}`
early return	loading and error screens	`if (!data) return <Spinner />`
variable	longer markup	`let content = ...; return content`
`null`	hide a whole component	`return isHidden ? null : <Panel />`

The zero trap

`{0 && <p>No items</p>}` renders a lone `0` on screen, because `0` is a valid React child. Compare explicitly instead:

JSX

```jsx
{count > 0 && <span>{count}</span>}
{items.length === 0 && <p>The list is empty.</p>}
```

Filtering and empty states

JSX

```
const visible = todos.filter((t) => !t.hidden);
if (visible.length === 0) return <p>Nothing to show.</p>;
return <ul>{visible.map(...)}</ul>;
```

6.3 Examples

Example 1: a list with keys.

JSX

```jsx
function People({ people }) {
  return (
    <ul>
      {people.map((person) => (
        <li key={person.id}>{person.name} - {person.city}</li>
      ))}
    </ul>
  );
}

const people = [
  { id: "a", name: "Ada", city: "London" },
  { id: "b", name: "Alan", city: "Cambridge" },
];
```

output

```
- Ada - London
- Alan - Cambridge
```

Example 2: filtering and an empty state.

JSX

```jsx
function TodoList({ todos }) {
  const visible = todos.filter((t) => !t.done);
  if (visible.length === 0) {
    return <p>All done. Enjoy the rest of the day.</p>;
  }
  return (
    <>
      <p>{visible.length} remaining</p>
      <ul>
        {visible.map((t) => (
          <li key={t.id}>{t.text}</li>
        ))}
      </ul>
    </>
  );
}

const todos = [
  { id: 1, text: "Learn JSX", done: true },
  { id: 2, text: "Learn state", done: false },
];
```

output

```
1 remaining
- Learn state
```

After marking the second item done:

output

```
All done. Enjoy the rest of the day.
```

Example 3: a ternary for the second button.

JSX

```jsx
function SaveButton({ isSaving }) {
  return <button>{isSaving ? "Saving..." : "Save"}</button>;
}
```

output

```
Save        (isSaving = false)
Saving...   (isSaving = true)
```

Example 4: an early return for loading.

JSX

```jsx
function Profile({ user }) {
  if (user === null) return <p>Loading...</p>;
  return <h2>{user.name}</h2>;
}
```

output

```
Loading...   (user === null)
Ada          (user = { name: "Ada" })
```

Example 5: the zero trap, fixed.

JSX

```jsx
function Badge({ count }) {
  return (
    <div>
      {count && <span>{count} new</span>}     {/* wrong: renders 0 */}
      {count > 0 && <span>{count} new</span>} {/* right */}
    </div>
  );
}
```

With `count={0}` the first line prints a stray `0`; the second prints nothing. With `count={3}` both print `3 new`.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Index keys on a reorderable or filtered list	state lands on the wrong row	use the item's id
`{count && <Badge />}` with `count = 0`	a stray `0` appears	`count > 0 && ...`

Mistake	Why it hurts	Best practice
Nested ternaries three levels deep	unreadable	early returns or variables
Mutating the array then keeping the same reference	no re-render	build a new array
No empty state	the screen looks broken	render a message
Forgetting `key` on the outer element of each item	React warns and mis-matches	put the key on the returned element

- `key` goes on the element you return from `map`, not on a child inside it.
- Render a list the way you describe it: filter, map, and handle the empty case.

6.5 Practice exercises

1. Why does React need a `key` for each item of a rendered list?
2. Which key would you choose for a list of messages fetched with ids, and why?
3. What is wrong with `{items.length && <p>Empty</p>}`?
4. Rewrite a nested ternary with an early return.
5. A list shows "no rows" after a filter removes everything. What should you render?

Solutions

1. Keys give each item a stable identity so React can match old and new children and keep component state on the right row.
2. `message.id`: it is unique among siblings and stable when the list changes.
3. When the length is `0`, `0 && ...` evaluates to `0`, which React renders as text; write `items.length === 0 && <p>Empty</p>`.
4. Example: `if (status === "loading") return <Spinner />; if (status === "error") return <Error />; return <Data data={data} />;`
5. An empty state, such as "No messages match this filter", so the user knows the area is intentionally blank.

6.6 Mini test

1. Which method turns an array into elements? A) `forEach` B) `map` C) `reduce` D) `filter` alone
2. Which key is the best choice for a list of users that can be filtered? A) the array index B) `Math.random()` C) the user's id from the data D) the user's name
3. What does `{0 && <Badge />}` render? A) Nothing B) The text `0` C) An empty badge D) It throws
4. Which pattern handles a loading screen most cleanly? A) `isLoading && <Spinner />` B) `isLoading ? <Spinner /> : <Content /> ? <Data /> : null` C) an early `return <Spinner />` before the main markup D) setting `innerHTML`

5. Where does the `key` prop belong? A) On the outermost element returned by the `map` callback B) On the array itself C) On the `ul` D) On each child inside the returned element
6. True or false: a component can read its own `key` through props.

Answer key and explanations

1. B - `map` returns a new array of elements, one per item.
2. C - a data id is unique and stable; indexes move when the list changes.
3. B - `0` is a valid React child, so it is rendered as text.
4. C - an early return avoids nesting and keeps one branch per return.
5. A - React uses the key of the element the callback returns.
6. False - keys are consumed by React itself and never reach props.

Chapter 7 - Forms and controlled inputs

7.1 Summary

- A controlled input keeps its value in React state and reports changes with `onChange`.
- The pair `value` plus `onChange` is what makes an input controlled.
- One state object per form scales better than many separate `useState` calls.
- The `name` attribute lets a single handler update the right field.
- `event.preventDefault()` stops the reload when the form is submitted.
- `textarea` and `select` are controlled exactly like `input`.
- Checkboxes and radio buttons use `checked` and `onChange`, not `value`.
- Simple validation can be computed during render.
- Trim and normalise values before storing or submitting them.
- An uncontrolled input with a ref is the exception, not the rule.

7.2 Concepts

The controlled loop

output

```
   value={form.email}
+--------+          +--------+
| state  |  -----> | <input>  --onChange-->  setForm({ ...form, email })
+--------+          +--------+                     |
   ^                                               |
   +-------------- one new state object -----------+
```

Because React owns the value, the screen and the state can never disagree.

One field or many

Approach	Good for
one `useState` per field	a single search box
one object per form	any form with three or more fields

The input types

Element	Value prop	Notes
`input type="text"`	`value` + `onChange`	the standard case
`textarea`	`value` + `onChange`	never use `children` for the text
`select`	`value` + `onChange`	options are children
`input type="checkbox"`	`checked` + `onChange`	boolean
`input type="radio"`	`checked` + `onChange`	grouped by `name`

Submitting

Put `onSubmit` on the form, not on the button, so the Enter key works too:

JSX

```jsx
function handleSubmit(event) {
  event.preventDefault();
  const trimmed = { ...form, email: form.email.trim() };
  console.log(trimmed);
}
```

Validation

Derive the errors while rendering, and show them next to the field:

JSX

```jsx
const emailError = form.email.includes("@") ? null : "Enter a valid email";
```

Strategy	Trade-off
on submit	quiet while typing, late feedback
per keystroke	instant, but shouts at half-typed values
on blur	balanced: check when leaving the field

7.3 Examples

Example 1: one controlled input.

JSX

```
function NameField() {
  const [name, setName] = useState("");
  return (
    <label>
      Name:
      <input value={name} onChange={(e) => setName(e.target.value)} />
    </label>
  );
}
```

Typing `Ada` shows, after each keystroke:

output

```
(empty) -> A -> Ad -> Ada
```

Example 2: a form object with one handler.

JSX

```
function Signup() {
  const [form, setForm] = useState({ email: "", plan: "free" });

  function handleChange(event) {
    const { name, value } = event.target;
    setForm({ ...form, [name]: value });
  }
  function handleSubmit(event) {
    event.preventDefault();
    console.log(form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" value={form.email} onChange={handleChange} />
      <select name="plan" value={form.plan} onChange={handleChange}>
        <option value="free">Free</option>
        <option value="pro">Pro</option>
      </select>
      <button type="submit">Sign up</button>
    </form>
  );
}
```

Typing `ada@example.com`, choosing `Pro` and submitting prints:

output

```
{ email: "ada@example.com", plan: "pro" }
```

The page does not reload.

Example 3: a checkbox.

JSX

```
function Terms() {
  const [accepted, setAccepted] = useState(false);
  return (
    <form>
      <label>
        <input
          type="checkbox"
          checked={accepted}
          onChange={(e) => setAccepted(e.target.checked)}
        />
        I accept the terms
      </label>
      <button disabled={!accepted}>Continue</button>
    </form>
  );
}
```

The button is disabled until the box is checked, then:

output

```
(button enabled)
```

Example 4: validation while rendering.

JSX

```
function EmailForm() {
  const [email, setEmail] = useState("");
  const error = email === "" || email.includes("@") ? "" : "That is not an email";

  return (
    <form>
      <input value={email} onChange={(e) => setEmail(e.target.value)} />
      <p style={{ color: "red" }}>{error}</p>
    </form>
  );
}
```

Typing `ada@` then `ada@example.com`:

output

```
That is not an email
(nothing, the message disappears)
```

Example 5: resetting a form.

JSX

```jsx
function Comment() {
  const empty = { text: "" };
  const [form, setForm] = useState(empty);

  function handleSubmit(event) {
    event.preventDefault();
    console.log(`sent: ${form.text}`);
    setForm(empty);           // back to the initial state
  }

  return (
    <form onSubmit={handleSubmit}>
      <textarea
        name="text"
        value={form.text}
        onChange={(e) => setForm({ text: e.target.value })}
      />
      <button type="submit">Send</button>
    </form>
  );
}
```

After typing `hello` and pressing Send:

output

```
sent: hello
(the textarea is empty again)
```

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`value` without `onChange`	the input is frozen, React warns	always pair them
`value={form.email}` when the field starts as `undefined`	React flips between controlled and uncontrolled	initialise with `""`
Submitting without `preventDefault`	the page reloads and the state is lost	call it first
Setting state from a checkbox's `value`	you store a string, not a boolean	use `checked` and `e.target.checked`
Storing raw keystrokes forever	junk data in the store	trim and normalise on submit
Resetting with `form.reset()`	React state no longer matches the screen	set the state back to the initial object

- One handler for the whole form, keyed by `event.target.name`.
- Show validation next to the field, and keep it short and specific.

7.5 Practice exercises

1. Which two props make an input controlled?
2. Why should `onSubmit` sit on the `<form>` rather than the button?
3. Write the setter call that updates only the `email` field of a form object.
4. Read the checked state of a checkbox in its handler.
5. What happens if a controlled input's value is `undefined`?

Solutions

1. `value` and `onChange`.
2. Pressing Enter in a field submits the form, and both buttons and the Enter key then trigger the same handler.
3. `setForm({ ...form, email: e.target.value.trim() })`.
4. `event.target.checked`.
5. React treats the input as uncontrolled, warns when switching, and the field may behave inconsistently; initialise with `""`.

7.6 Mini test

1. Which pair makes an input controlled? A) `name` and `id` B) `value` and `onChange` C) `defaultValue` and `onBlur` D) `ref` and `type`
2. Which attribute should a multi-field handler read from the event? A) `event.target.id` B) `event.target.name` C) `event.target.className` D) `event.target.type`
3. How is a checkbox controlled? A) `value` and `onChange` B) `checked` and `onChange` C) `selected` and `onChange` D) `defaultChecked`
4. Which call must come first in a submit handler? A) `setForm({})` B) `event.preventDefault()` C) `event.stopPropagation()` D) `console.log`
5. True or false: `<textarea value={text} onChange={...}>` needs children holding the text.
6. How do you empty a form after a successful submit? A) `document.querySelector("form").reset()` B) set the state back to the initial value C) `form.value = ""` D) reload the page

Answer key and explanations

1. B - React owns the value and learns about changes through the handler.
2. B - the `name` attribute identifies which field changed.
3. B - checkboxes store a boolean in `checked`/`e.target.checked`.
4. B - otherwise the browser submits the form and the page reloads.
5. False - `children` are ignored; the `value` prop carries the text.
6. B - state is the single source of truth, so resetting state resets the screen.

Chapter 8 - Side effects with useEffect

8.1 Summary

- `useEffect(setup, deps)` runs after the component is painted on screen.
- Effects synchronise a component with systems React does not control: timers, network, DOM APIs, external widgets.
- The dependency array tells React when to run the effect again.
- A cleanup function returned from the effect runs before the next run and on unmount.
- An empty dependency array means "run once after the first render".
- Omitting the array means the effect runs after every render, which is rarely what you want.
- Effects cannot be `async`; the asynchronous work is defined inside the effect.
- In development, Strict Mode runs effects twice on purpose, to expose missing cleanup.
- An effect must not return anything except the cleanup function.
- Actions caused directly by the user belong in event handlers, not in effects.

8.2 Concepts

Where each kind of code belongs

Code	Where
reacting to a click	the event handler
transforming data for the render	during render, in the component body
talking to timers, network, browser APIs, third-party widgets	an effect

Anatomy

JSX

```jsx
useEffect(() => {
  // setup: start something
  const id = setInterval(tick, 1000);
  return () => {
    // cleanup: stop it again
    clearInterval(id);
  };
}, []); // dependencies
```

The three dependency shapes

Dependencies	Effect runs
none written	after every render
`[]`	once, after the first render
`[a, b]`	after the first render, and whenever `a` or `b` changes

React compares dependencies with `Object.is`, so a new object or array literal in the list is a dependency "change" on every render.

Cleanup

The setup starts	The cleanup must
an interval or timeout	`clearInterval` / `clearTimeout`
a DOM listener	`removeEventListener`
a subscription	`unsubscribe`
a fetch	`controller.abort()`

Strict Mode

In development the templates mount components twice, so every effect runs once and is immediately cleaned up. This reveals effects that forget to clean up.

Stale values and loops

If an effect sets state that the effect itself depends on, it schedules a new render, which runs the effect again: an endless loop. Fix the dependencies, or move the update out of the effect.

8.3 Examples

Example 1: the document title.

JSX

```jsx
import { useEffect } from "react";

function DocumentTitle({ title }) {
  useEffect(() => {
    document.title = title;
  }, [title]);

  return null;
}

<DocumentTitle title="Inbox (2)" />
```

output

```
(the browser tab shows "Inbox (2)"; changing the prop updates it)
```

Example 2: a timer that stops with the component.

JSX

```jsx
function Clock() {
  const [now, setNow] = useState(() => new Date());

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id);
  }, []);

  return <p>{now.toLocaleTimeString()}</p>;
}
```

output

```
21:04:31
21:04:32
(after the component unmounts nothing appears any more: the interval is cleared)
```

Example 3: a window listener.

JSX

```jsx
function WindowSize() {
  const [size, setSize] = useState({
    w: window.innerWidth,
    h: window.innerHeight,
  });

  useEffect(() => {
    function handleResize() {
      setSize({ w: window.innerWidth, h: window.innerHeight });
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <p>{size.w} x {size.h}</p>;
}
```

output

```
1440 x 900
(after resizing the window, e.g. 1200 x 800)
```

Example 4: an effect that depends on a value.

JSX

```jsx
function UserPanel({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    let ignore = false;
    fetchUser(userId).then((data) => {
      if (!ignore) setUser(data);
    });
    return () => { ignore = true; };
  }, [userId]);

  if (user === null) return <p>Loading...</p>;
  return <p>{user.name}</p>;
}
```

With `userId = 1` then `userId = 2`:

output

```
Loading...
Ada
(then, after the prop changes)
Alan
```

The `ignore` flag discards the answer of the request that belongs to the old user.

Example 5: the Strict Mode double run.

JSX

```jsx
useEffect(() => {
  console.log("setup");
  return () => console.log("cleanup");
}, []);
```

In development with `StrictMode`:

output

```
setup
cleanup
setup
```

The effect is mounted, immediately unmounted and mounted again. Production logs it once.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Forgetting cleanup	intervals and listeners leak after unmount	return the cleanup function
`useEffect(async () => ...)`	React receives a promise instead of a cleanup	define an inner async function
Setting state the effect depends on	infinite loop of renders	fix the dependency list
Using an effect for derived data	an extra render and a flicker	compute during render
Fetching without cancellation	slow old answers overwrite new ones	`AbortController` (chapter 9)
Object literals in the dependency array	the effect re-runs on every render	primitives, or memoise the value

JSX

```jsx
// the loop: this effect changes its own dependency
useEffect(() => {
  setCount(count + 1);
}, [count]);
```

- Ask "is this caused by rendering, or by the user?" Rendering means an effect, the user means an event handler.
- Include every prop and state value the effect reads in its dependency list.

8.5 Practice exercises

1. When exactly does the setup function of an effect run?
2. What does a dependency array of `[]` mean?
3. Where does the cleanup function go, and when is it called?
4. Why can the callback of `useEffect` not be declared `async`?
5. A user action clicks a button and saves data. Effect or event handler?

Solutions

1. After React has painted the component, once the render is committed.
2. The effect runs once after the first render and never again.
3. It is returned by the setup function, and React calls it before the next run of the effect and when the component unmounts.
4. An `async` function always returns a promise, and React would treat that promise as a cleanup function; the async work is defined inside instead.
5. An event handler: it happens because of the user, not because of rendering.

8.6 Mini test

1. When does an effect's setup function run? A) During render, before the JSX is returned B) After the render is painted C) Before the component function is called D) Only on unmount
2. What does an empty dependency array do? A) Runs the effect after every render B) Runs the effect never C) Runs the effect once after the first render D) Runs it only in development
3. Which value must the effect callback return? A) A promise B) Nothing or a cleanup function C) The next state D) A boolean
4. What happens in development under Strict Mode? A) Effects are disabled B) Setup and cleanup each run twice C) The state is cleared D) The build fails
5. True or false: setting the state that an effect depends on inside that effect is safe as long as the list is empty.
6. An interval started in an effect must be cleared where? A) In the component body B) In the cleanup function the effect returns C) In a click handler D) Never

Answer key and explanations

1. B - effects run after the commit, so the DOM already exists.
2. C - `[]` means "no dependencies ever change".
3. B - anything else, such as a promise from an async function, is an error.

4. B - the double run exposes missing cleanup.
5. False - an empty list means the effect runs once, but the dependency must still be handled correctly; a self-triggered update is the classic loop.
6. B - the cleanup runs on unmount and before every re-run.

Chapter 9 - Data fetching patterns

9.1 Summary

- Data fetching belongs in an effect because it must start after the first paint.
- A fetching component tracks three things: the data, a loading flag and an error.
- An `AbortController` cancels the previous request when the inputs change.
- Race conditions let a slow old response overwrite a newer one.
- An effect cannot be `async`; the asynchronous work is defined inside the effect.
- `Promise.all` runs independent requests together instead of one after another.
- A failed fetch needs a message and a way to retry, not a blank screen.
- Never fetch during render; there is no place to put the result.
- Libraries such as TanStack Query add caching, retries and freshness checking.
- A loading placeholder should have roughly the shape of the content it replaces.

9.2 Concepts

The request lifecycle

output

```
idle --> loading --> success
                  \--> error --> (retry) --> loading
```

The state shape

State	Type	Initial
`data`	the payload or `null`	`null`
`isLoading`	boolean	`false`, or `true` when the first fetch starts at mount
`error`	message or `null`	`null`

Race conditions

output

```
t=0    request for query "ab"  ................. arrives at 300 ms
t=10   request for query "abc" .... arrives at  40 ms
The screen shows "abc", then the slow "ab" answer overwrites it: stale data.
```

Cancelling with `AbortController`, or ignoring results from superseded requests, fixes this.

Cancellation anatomy

JSX

```jsx
useEffect(() => {
  const controller = new AbortController();
  fetch(url, { signal: controller.signal })
    .then((res) => res.json())
    .then(setData)
    .catch((err) => {
      if (err.name !== "AbortError") setError(err.message);
    });
  return () => controller.abort();
}, [url]);
```

`controller.abort()` makes the pending `fetch` promise reject with an `AbortError`, which the handler ignores.

When to add a library

Situation	Good choice
one simple screen	`fetch` in an effect
the same data needed in several places	TanStack Query or SWR
caching, retries, background refresh	TanStack Query
a server framework with loaders	the framework's loader

9.3 Examples

Example 1: the three states.

JSX

```jsx
function Users() {
  const [users, setUsers] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("https://api.example.com/users")
      .then((res) => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then((data) => setUsers(data))
      .catch((err) => setError(err.message))
      .finally(() => setIsLoading(false));
  }, []);

  if (isLoading) return <p>Loading...</p>;
  if (error !== null) return <p style={{ color: "red" }}>Error: {error}</p>;

  return (
    <ul>
      {users.map((u) => (
        <li key={u.id}>{u.name}</li>
      ))}
    </ul>
  );
}
```

output

```
Loading...                    (while the request is in flight)
- Ada                         (on success)
- Alan
Error: HTTP 500               (on failure)
```

Example 2: cancelling the previous request.

JSX

```
function Profile({ userId }) {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setIsLoading(true);
    fetch(`https://api.example.com/users/${userId}`, {
      signal: controller.signal,
    })
      .then((res) => res.json())
      .then((data) => { setUser(data); setError(null); })
      .catch((err) => {
        if (err.name !== "AbortError") setError(err.message);
      })
      .finally(() => setIsLoading(false));

    return () => controller.abort();
  }, [userId]);

  if (isLoading) return <p>Loading...</p>;
  if (error !== null) return <p>Error: {error}</p>;
  return <h2>{user.name}</h2>;
}
```

Switching quickly from user 1 to user 2 always ends with:

output

```
Alan
```

Only the newest result is ever shown.

Example 3: two requests in parallel.

JSX

```
function Dashboard() {
  const [data, setData] = useState(null);

  useEffect(() => {
    Promise.all([
      fetch("/api/user").then((r) => r.json()),
      fetch("/api/alerts").then((r) => r.json()),
    ])
      .then(([user, alerts]) => setData({ user, alerts }))
      .catch((err) => console.error(err));
  }, []);

  if (data === null) return <p>Loading...</p>;
  return <p>{data.user.name}: {data.alerts.length} alerts</p>;
}
```

output

```
Loading...
Ada: 3 alerts        (both responses arrived; the total wait is one round trip)
```

Example 4: a retry button.

JSX

```jsx
function Report() {
  const [attempt, setAttempt] = useState(0);
  const [status, setStatus] = useState("loading");

  useEffect(() => {
    setStatus("loading");
    fetch("/api/report")
      .then((res) => {
        if (!res.ok) throw new Error("failed");
        return res.json();
      })
      .then(() => setStatus("ok"))
      .catch(() => setStatus("error"));
  }, [attempt]);

  if (status === "loading") return <p>Loading...</p>;
  return (
    <div>
      {status === "error" && <p>Something went wrong.</p>}
      <button onClick={() => setAttempt((a) => a + 1)}>Retry</button>
    </div>
  );
}
```

output

```
Something went wrong. [Retry]
(clicking Retry runs the effect again and shows Loading... then the report)
```

Example 5: the race condition, made visible.

JSX

```jsx
function Search({ query }) {
  const [results, setResults] = useState([]);

  useEffect(() => {
    search(query).then((r) => setResults(r)); // no cancellation
  }, [query]);

  return <ResultsList results={results} />;
}
```

output

```
typing "ab" then "abc": the screen may briefly show the results of "ab"
after the results of "abc", because the slower request is not discarded.
```

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
No loading flag	old or blank content looks like a bug	track `isLoading`
Ignoring errors	silent failure, the user sees nothing	error state plus a retry
Fetching without cancellation	race conditions and wasted work	`AbortController`

Mistake	Why it hurts	Best practice
`useEffect(async () => ...)`	React rejects a promise as the return value	inner async function
Fetching during render	no place to put the result, infinite loop	effect or a data library
Sequential fetches for independent data	two waits instead of one	`Promise.all`

- React to `res.ok === false` by throwing; a `fetch` only rejects on network failure, not on a 404 or a 500.
- Keep the request inside the effect that depends on its parameters.

9.5 Practice exercises

1. Which three pieces of state does a fetching component need?
2. Why is fetching usually done in an effect rather than during render?
3. How do you cancel a fetch when the component unmounts?
4. What does the cleanup of Example 2 guarantee for a fast user?
5. How do you fetch two independent endpoints with one round trip?

Solutions

1. `data`, `isLoading` and `error`.
2. Fetching is asynchronous and touches an external system; render must be synchronous and pure, and the result of the fetch has no place to live yet.
3. Create an `AbortController`, pass its `signal` to `fetch`, and call `controller.abort()` in the effect's cleanup.
4. The response of the superseded request is aborted, so a slow old answer can never overwrite the newer one.
5. Start both promises together and settle them with `Promise.all`.

9.6 Mini test

1. Which state trio belongs to a fetching component? A) `props`, `state`, `refs` B) `data`, `isLoading`, `error` C) `count`, `name`, `flag` D) `query`, `page`, `sort`
2. What does `fetch` reject on? A) any non-2xx response B) a network failure only C) a slow response D) JSON parse success
3. Which object cancels a request? A) `AbortController` B) `CancelToken` C) `EventEmitter` D) `XMLHttpRequest`
4. How is a slow earlier response prevented from overwriting a newer one? A) by sorting the responses B) by aborting previous requests or ignoring stale results C) by caching everything D) by awaiting each request

5. True or false: `useEffect(() => { ... }, [])` can be written as `useEffect(async () => { ... }, [])`.
6. Which API runs two independent fetches in parallel? A) `Promise.race` B) `Promise.all` C) `Promise.then` D) `await fetch(); await fetch();`

Answer key and explanations

1. B - those three drive the loading, error and success branches.
2. B - `fetch` only rejects for network-level failures; HTTP errors need a manual check of `res.ok`.
3. A - its `signal` is passed to `fetch` and `abort()` is called in cleanup.
4. B - stale answers are cancelled or discarded.
5. False - an async callback returns a promise; the async work must be defined inside the effect.
6. B - `Promise.all` starts both at once and waits for both.

Chapter 10 - Refs, context, custom hooks and performance

10.1 Summary

- `useRef` keeps a mutable box whose changes do not trigger a render.
- The main use of a ref is reaching a DOM node through the `ref` prop.
- Context shares a value with a whole subtree without passing props at every level.
- Every consumer re-renders when a context value changes, so providers stay narrow.
- `memo` skips re-rendering a component whose props have not changed.
- `useMemo` caches a computed value and `useCallback` caches a function.
- A custom hook extracts stateful logic and its name starts with `use`.
- Hooks compose: a custom hook can call other hooks, including React's.
- Memoisation comes after measuring, not before.
- Beyond these basics the ecosystem offers routing, state libraries and testing.

10.2 Concepts

Refs

JSX

```jsx
const inputRef = useRef(null);
return <input ref={inputRef} />;       // inputRef.current is the DOM node
```

A ref object is stable for the life of the component. Mutating `current` never re-renders, which makes refs the right tool for:

Use	Example
DOM access	`inputRef.current.focus()`
timers	`idRef.current = setInterval(...)`
previous values	`prevRef.current = value` during render

Context

JSX

```jsx
const ThemeContext = createContext("light");

function App() {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={theme}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const theme = useContext(ThemeContext);
  return <p>{theme}</p>;
}
```

Rule	Why
one provider per concern	a provider re-renders all its consumers
memoise the value	a fresh object literal changes every render
keep the tree shallow? context skips levels, so depth does not matter	the cost is in the number of consumers
reach for props first	props make data flow visible

Memoisation

Tool	Caches	Recomputes when
`memo(Component)`	the rendered output	the props change
`useMemo(() => value, deps)`	a value	the dependencies change
`useCallback(fn, deps)`	a function	the dependencies change

Memoisation only pays off when the work is genuinely expensive or the child receives stable props. Recent React versions can also apply this automatically through the React Compiler, which removes most hand-written memoisation.

Custom hooks

JSX

```jsx
function useToggle(initial) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn((v) => !v), []);
  return [on, toggle];
}
```

Rules:

1. The name starts with `use`, so React knows it contains hooks.
2. It is called at the top level of a component or of another hook.
3. It returns whatever its consumer needs: values, functions, objects.

What comes next

Topic	Where
routing	reactrouter.com
server data and caching	TanStack Query
global state	Redux, Zustand, or context plus a reducer
testing	Vitest with Testing Library
TypeScript	react.dev/learn/typescript

10.3 Examples

Example 1: focusing an input.

JSX

```jsx
import { useEffect, useRef } from "react";

function AutoFocus() {
  const inputRef = useRef(null);

  useEffect(() => {
    inputRef.current.focus();
  }, []);

  return <input ref={inputRef} placeholder="Type here" />;
}
```

output

```
(the field has focus as soon as it appears; document.activeElement is the input)
```

Example 2: context for a theme.

JSX

```jsx
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext("light");

function App() {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={theme}>
      <button
        onClick={() => setTheme(theme === "light" ? "dark" : "light")}
      >
        Toggle
      </button>
      <Panel />
    </ThemeContext.Provider>
  );
}

function Panel() {
  const theme = useContext(ThemeContext);
  return <p>Current theme: {theme}</p>;
}
```

output

```
Current theme: light
(one click)
Current theme: dark
```

Example 3: a custom hook.

JSX

```jsx
import { useEffect, useState } from "react";

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const stored = window.localStorage.getItem(key);
    return stored === null ? initial : JSON.parse(stored);
  });

  useEffect(() => {
    window.localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

function NameBox() {
  const [name, setName] = useLocalStorage("name", "");
  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

output

```
(typing a name and reloading the page shows the same value again)
```

Example 4: memo with a stable function.

JSX

```jsx
import { memo, useCallback, useState } from "react";

const Item = memo(function Item({ label, onSelect }) {
  console.log("Item rendered");
  return <li onClick={() => onSelect(label)}>{label}</li>;
});

function List() {
  const [count, setCount] = useState(0);
  const [selected, setSelected] = useState(null);
  const handleSelect = useCallback((label) => setSelected(label), []);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Rerender ({count})</button>
      <Item label="First" onSelect={handleSelect} />
      <p>{selected}</p>
    </div>
  );
}
```

output

```
Item rendered        (once, on mount)
(clicking "Rerender" several times logs nothing new)
First                (clicked by the user, from the stable handler)
```

Without `useCallback` every click would create a new function, `memo` would see changed props, and the item would log on every re-render.

Example 5: `useMemo` for expensive work.

JSX

```jsx
function Sorted({ items, query }) {
  const visible = useMemo(
    () => items.filter((i) => i.includes(query)).sort(),
    [items, query]
  );

  return (
    <ul>
      {visible.map((i) => (
        <li key={i}>{i}</li>
      ))}
    </ul>
  );
}

const items = ["pear", "apple", "plum"];
```

With `query="p"`:

output

```
- pear
- plum
```

Without `useMemo` the same list is produced, but the filter and sort run on every render, including those caused by unrelated state.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
A ref for every value	nothing re-renders when it changes	use state unless the value must not render
One context for the whole app	every consumer re-renders on any change	narrow, per-concern providers
`memo` with inline props	the optimisation never triggers	`useCallback`/`useMemo`, or drop `memo`
A custom hook without the `use` prefix	lint errors and confusion	always `useX`
Memoising cheap renders	complexity with no gain	measure first
Reading `ref.current` during render	the value is not reactive yet	read it in effects or handlers

- Profile before optimising: most applications are fast without memoisation.
- Convert a context value into a memoised object when it holds several fields.

10.5 Practice exercises

1. What does `useRef` return, and how does its value differ from state?
2. Where do you attach a ref to reach an input DOM node?
3. Why does changing a context value re-render all of its consumers?
4. Write the signature of a custom hook named `useCounter`.
5. When is `memo(Component)` worth adding?

Solutions

1. A stable object `{ current }`; writing to `current` does not schedule a render, while a state update does.
2. `ref={inputRef}` on the JSX element; `inputRef.current` is the node.
3. The provider re-renders with a new value, and every component that called `useContext` for that context re-renders as part of it.
4. `function useCounter(initial = 0) { const [count, setCount] = useState(initial); ... return [count, increment]; }`
5. When the component renders often with identical props, the render work is expensive, and the props can be kept stable.

10.6 Mini test

1. What does `useRef(null)` return? A) The DOM node itself B) A stable object with a `current` property C) A state value D) A cleanup function

2. Changing `ref.current` does what to the component? A) Re-renders it B) Nothing; refs are not reactive C) Runs the effects D) Clears the state
3. Which pair creates and reads a context? A) `createRef` and `readRef` B) `createContext` and `useContext` C) `useState` and `useEffect` D) `buildContext` and `Context.read`
4. What happens when a provider's value changes? A) Only direct children re-render B) Every consumer of that context re-renders C) The whole page reloads D) Nothing
5. What must the name of a custom hook start with? A) `use` B) `get` C) `custom` D) `hook`
6. True or false: `memo` helps even when the parent passes a new inline function on every render.

Answer key and explanations

1. B - the object identity never changes between renders.
2. B - that is the point: refs carry values React does not render.
3. B - one call creates the context object, the other reads its value.
4. B - all consumers re-render, so providers should stay narrow.
5. A - the `use` prefix marks the function as a hook.
6. False - a new function is a new prop, so `memo` sees a change and the work is repeated.

Cheat sheet

11.1 Commands

Command	Effect
`npm create vite@latest my-app -- --template react`	scaffold a project
`npm install`	install dependencies
`npm run dev`	dev server with hot reload
`npm run build`	production bundle in `dist/`
`npm run preview`	serve the production bundle

11.2 JSX

Rule	Correct form
one root element	`<> ... </>` or one parent
closed tags	`<img src="a.png" />`
class attribute	`className="card"`
label attribute	`htmlFor="email"`

Rule	Correct form
inline style	`style={{ color: "red" }}`
comment	`{/* note */}`
expression	`{value}`, `{a + b}`, `{cond ? x : y}`
statements	compute them before the `return`

11.3 Hooks

Hook	Purpose	Shape
`useState`	local state	`const [v, setV] = useState(0)`
`useEffect`	synchronise with the outside	`useEffect(() => () => {}, [deps])`
`useRef`	mutable, non-reactive box	`const r = useRef(null)`
`useContext`	read a context value	`const v = useContext(Ctx)`
`useMemo`	cache a value	`useMemo(() => x, [deps])`
`useCallback`	cache a function	`useCallback(fn, [deps])`
`memo`	skip re-render on equal props	`memo(function C(props){})`

11.4 State recipes

Task	Code
set a value	`setCount(5)`
increment	`setCount((c) => c + 1)`
object field	`setUser({ ...user, name: "Ada" })`
add to array	`setItems([...items, item])`
remove from array	`setItems(items.filter((i) => i.id !== id))`
reset	`setForm(initialForm)`
derive	compute during render, never store

11.5 Events

Need	Write
click	`<button onClick={handleClick}>`
click with argument	`onClick={() => remove(id)}`

Need	Write
input change	`onChange={(e) => setText(e.target.value)}`
submit	`<form onSubmit={handleSubmit}>` + `event.preventDefault()`
child to parent	prop `onX`, parent handler `handleX`
key pressed	`onKeyDown` and `event.key`

11.6 Lists and conditionals

Need	Write
list	`{items.map((i) => <li key={i.id}>{i.text}</li>)}`
show if true	`{ok && <p>Fine</p>}`
guard zero	`{count > 0 && <span>{count}</span>}`
either/or	`{open ? <Menu /> : <Button />}`
loading branch	`if (loading) return <Spinner />`
render nothing	`return null`
empty list	`{items.length === 0 && <p>Empty</p>}`

11.7 Forms

Element	Props
`input type="text"`	`value` + `onChange`
`textarea`	`value` + `onChange` (no children)
`select`	`value` + `onChange`
`checkbox` / `radio`	`checked` + `onChange`
one handler	read `event.target.name` and `event.target.value`
submit	`event.preventDefault()`, then trim and send

11.8 Effects

Ds	Runs
none	after every render
`[]`	once after mount
`[a, b]`	on mount and when `a` or `b` change

Setup	Cleanup
`setInterval`	`clearInterval`
`addEventListener`	`removeEventListener`
subscription	`unsubscribe`
`fetch(url, { signal })`	`controller.abort()`

11.9 Quick answers

Question	Answer
Where to fetch	in an effect, with `AbortController`
Fetch states	`data`, `isLoading`, `error`
Parallel requests	`Promise.all`
Focus a DOM node	`ref` + `current.focus()` in an effect
Share a value deeply	context, with a memoised value
Custom hook name	starts with `use`, called at the top level
Optimise only after	measuring, with `memo`/`useMemo`/`useCallback`

Final test

1. What does the JSX syntax compile to? A) Plain HTML text B) JavaScript function calls that create objects C) XML nodes D) CSS rules
2. Which attribute name is valid in JSX? A) `class` B) `for` C) `className` D) `onclick`
3. Which hook adds a piece of state to a function component? A) `useEffect` B) `useRef` C) `useState` D) `useContext`
4. True or false: hooks may be called inside an `if` statement as long as the condition is stable.
5. Which pair of calls reliably applies two increments from one click? A) `setCount(count + 1); setCount(count + 1);` B) `setCount((c) => c + 1); setCount((c) => c + 1);` C) `count += 2; setCount(count);` D) `setCount(count++);`
6. Why must an object in state be replaced instead of mutated?
7. Which prop handles a click in React? A) `onclick` B) `onClick` C) `clickHandler` D) `onPress`
8. What does `event.preventDefault()` do inside a submit handler?
9. Which prop does every item of a rendered list need? A) `id` B) `key` C) `ref` D) `index`
10. Which key is best for a list of tasks that can be reordered and filtered? A) the array index B) `Math.random()` C) the task's id from the data D) the task text

11. What does `{0 && <Badge />}` render? A) Nothing B) The text `0` C) An empty badge D) It throws an error
12. Which two props make a text input controlled? A) `name` and `id` B) `value` and `onChange` C) `defaultValue` and `onBlur` D) `ref` and `type`
13. How is a checkbox controlled? A) with `value` and `onChange` B) with `checked` and `onChange` C) with `selected` and `onChange` D) with `defaultChecked` only
14. When does the setup function of `useEffect` run? A) during render, before the JSX is returned B) after the component is painted C) before the component mounts D) only when the component unmounts
15. What is the purpose of the function an effect returns?
16. Which dependency array makes an effect run after every render? A) `[]` B) the argument is omitted entirely C) `[deps]` D) `{}`
17. True or false: `useEffect(async () => { ... }, [])` is the recommended way to fetch data.
18. Which object cancels a pending fetch when its inputs change? A) `AbortController` B) `CancelToken` C) `EventEmitter` D) `PromiseSignal`
19. What does `useRef` return, and what happens when you write to its `current` property?
20. How do you move focus to an input right after the component appears?
21. Which problem does context solve?
22. What does `memo(Component)` skip when the props are shallowly equal?
23. What prefix must a custom hook's name have, and where may it be called?
24. What is the difference between `useMemo` and `useCallback`?
25. Write the JSX for a button whose click removes the item with id 3.

12.1 Final test answers

1. B - the build turns JSX into `jsx`/`createElement` calls that produce element objects.
2. C - React uses prop names: `className`, `htmlFor`, camelCase events.
3. C - `useState` returns the current value and the setter.
4. False - hooks must be called at the top level, in the same order, on every render.
5. B - queued updaters each receive the latest value, so both increments apply.
6. React detects change by reference with `Object.is`; a mutated object looks identical and the screen never updates, so a new object must be created.
7. B - event props are camelCase.
8. It cancels the browser's default reaction, so the form does not submit and reload the page, and React handles the event instead.
9. B - `key` gives React a stable identity for each item.
10. C - a data id stays with the item when the list changes order or filters; an index does not.
11. B - `0` is a valid React child, so `&&` yields the number itself; compare explicitly instead.
12. B - React owns the value and learns about changes through `onChange`.
13. B - checkboxes hold a boolean in `checked` and `event.target.checked`.

14. B - effects run after the commit and the first paint.
15. It is the cleanup: React calls it before the next run of the effect and when the component unmounts, to release listeners, timers and requests.
16. B - with no second argument there are no dependencies, so the effect runs after every render.
17. False - an async function returns a promise, which React cannot treat as a cleanup function; the async work is defined inside the effect.
18. A - its `signal` is passed to `fetch` and `abort()` is called in the cleanup.
19. A stable `{ current }` object; writing to `current` does not schedule a render, which is why refs are used for DOM access and non-reactive values.
20. Keep a ref on the input (`ref={inputRef}`) and call `inputRef.current.focus()` inside an effect that runs once.
21. Prop drilling: it shares a value with a whole subtree without passing it through every intermediate component.
22. The render: React reuses the previous output when the props are shallowly equal, so the component function is not called.
23. `use`, and it may be called only at the top level of a component or of another custom hook.
24. `useMemo` caches a computed value; `useCallback` caches the identity of a function.
25. `<button onClick={() => removeItem(3)}>Delete</button>` - the handler is a function, and the id is captured by the inline arrow.

Sources

Consulted while writing this volume:

- React, the learning path: https://react.dev/learn
- Quick start and editor setup: https://react.dev/learn/installation
- Creating a React project: https://react.dev/learn/start-a-new-react-project
- Your first component: https://react.dev/learn/your-first-component
- Writing markup with JSX: https://react.dev/learn/writing-markup-with-jsx
- JavaScript in JSX with curly braces: https://react.dev/learn/javascript-in-jsx-with-curly-braces
- Passing props to a component: https://react.dev/learn/passing-props-to-a-component
- Conditional rendering: https://react.dev/learn/conditional-rendering
- Rendering lists: https://react.dev/learn/rendering-lists
- Responding to events: https://react.dev/learn/responding-to-events
- State, a component's memory: https://react.dev/learn/state-a-components-memory
- Queueing a series of state updates: https://react.dev/learn/queueing-a-series-of-state-updates
- Updating objects in state: https://react.dev/learn/updating-objects-in-state

- Updating arrays in state: https://react.dev/learn/updating-arrays-in-state
- Synchronising with effects: https://react.dev/learn/synchronizing-with-effects
- You might not need an effect: https://react.dev/learn/you-might-not-need-an-effect
- Referencing values with refs: https://react.dev/learn/referencing-values-with-refs
- Passing data deeply with context: https://react.dev/learn/passing-data-deeply-with-context
- Reusing logic with custom hooks: https://react.dev/learn/reusing-logic-with-custom-hooks
- Rules of React: https://react.dev/reference/rules
- TypeScript with React: https://react.dev/learn/typescript
- Vite, the build tool used for the examples: https://vite.dev/guide/
- React Router: https://reactrouter.com/
- Testing Library for React: https://testing-library.com/docs/react-testing-library/intro/
- TanStack Query: https://tanstack.com/query/latest
- ReactPlay, the source of the exercise themes: https://reactplay.io/